Autonomous Software Agents Project Report

Daniele Buondonno
267888
daniele.buondonno@studenti.unitn.it

Sasha Pektovic
264689
sasha.pektovic@studenti.unitn.it

Abstract

Report for the Autonomous Software Agents project. The objective of the project is to develop two autonomous agents. One using the BDI (Belief-Desire-Intention) architecture, and one through LLM API calls. The agents' objective is to play the game Deliveroo.js https://deliveroojs.azurewebsites.net/. It is a game where the agents have to collect parcels and deliver them while managing different obstacles and performing actions in a competitive environment.

1 Introduction

In this report, we will give an overview of our project for the Autonomous Software Agents course. The project's goal is to create and develop autonomous agents that can play the game Deliveroo.js.
The game consists in picking up and deliverying parcels, which yield different reward values. The game's environment is competitive, as rival agents can be present. The project is divided into two parts.

- BDI Agent: An autonomous agent that uses the Belief-Desire-Intention architecture to observe and perform actions in an environment. This architecture allows the agent to maintain a set of beliefs about the environment obtained through sensing. Through the belief set, the architecture allows the agent to compute possible options (desires) along a utility score for them. The option with the best utility score is selected and performed, if possible.
  The architecture prescribes that a loop is present, where options continue to be generated given new observations, and where the current Intention can be revised, as new information might tell the agent that it cannot be performed anymore or that there's a better option to be performed.

- LLM Agent: An agent that acts as a partner to the BDI agent. It performs like the BDI agent through a setup composed of API calls and prompts. It must be able to understand the rules of the game and the environment around it.
  It must also be able to understand, evaluate and perform missions delivered to it through the game chat, which can yield positive or negative results.

We will first highlight the structure of the BDI agent, explaining its different components and operations. We will go through the data structures used, the strategies we decided to implement and our reasoning behind both. We will then explain the LLM's agent structure and operations. To conclude we will give some brief results and our thoughts for further improving our solutions.

2 Belief

In the Belief-Desire-Intention architecture, an agent's belief set represents its observations and beliefs over the environment. The term belief is not used casually, as what's 'contained' in its belief set isn't a ground truth, but what the agent has deducted from both present and past observations, which might not be true anymore.
In the Deliveroo game, observations are done through sensing events propagated by the game server. As we will see, in our solution we try to keep the most up to date informations and forget what we confirm to be outdated, so priority is given to the information we're most confident about. We will now go over the different classes implementing the agent's Belief Set.

2.1 Me

This class holds the agent's identity, its position and its score. For the agent's position, we decided to gate non integer values, as they indicate that the agent is attempting to move. This avoids inconsistent information about the position and avoids making wrong assumptions about a succesful move through the environment. To balance this out, as soon as a move action is succesful, we update its position to the new coordinates.

2.2 Parcels

This class holds all the data structures and logic used to represent and update the agent's belief about the parcels it's carrying or that it has observed, during both past and current sensing events. To hold these informations, we use three maps.

2.2.1 Visible

In this map we store all the parcels the agents saw during the latest sensing, indexed by their id. This map is rebuilt every time the agent performs sensing. It is particularly useful to recognise which parcels the agent is most confident about, as it can currently see them.

2.2.2 Known

This map contains all 'known' parcels, indexed by their id. A known parcel is a parcel that has been observed through previous sensings. Along with the parcels data, we store a timestamp indicating when they've been observed. This is particularly useful for computing their utility by estimating their decayed value, if parcel reward decay is currently enabled by the server.
To maintain this data structure as accurate as possible, whenever a known parcel's location is observed, and the parcel is not present, it is removed and forgotten by the agent. If the agent has a partner, this map also holds the information of the parcels reported by the partner.

2.2.3 Carried

This simple map holds the information about the parcels the agent's currently carrying, indexed by their id.

2.3 Crates

This class is specifically designed to represent the agent's beliefs about the crates in the map, if present, and to update such informations. A 'known' map is also used here, it works the same way as for the parcels, only this time it stores crates information. We also made use of a

'byPosition' map as a support structure to make operations easier to code and more efficient, as it indexes crates by their position string.

2.4 Agents

This class holds the agent's beliefs about the other agents perceived in the game environment and the logic to update them. We used a map to store these informations.

2.4.1 Others

This map contains the information about the agents perceived through the latest sensing event, indexed by their id.
During the update, we store their positions regardless if they're an integer or fractional. This is done on purpose as a fractional coordinate tells the agent that the rival agent is attempting to move, so two tiles are blocked instead of one at that moment.
In fact, if a rival agent is not moving, we only consider its current position as blocked. If it is attempting to move we instead consider its current position and the position its trying to move to as blocked.

2.5 Partner

This class is designed to represent and handle the information about the agent's partner, if there is one. It also holds the information the agent might want to share with its partner.
In particular the two agents share the information about the parcels they're currently carrying, that they're currently observing and the parcel they want to commit to. This helps avoid cases where both agents end up pursuing the same parcel, and it also allows them to evaluate which of the two should pursue a given parcel. The agents can in fact claim a parcel for their own by first evaluating if they're in a better spot to take it compared to the partner.

2.6 World

This class holds the agent's beliefs about the game's world. It stores the data regarding the game's map, configuration and metadata sent out by the server upon connecting or when a change about the config is made. It also contains the set of visible positions observed by the agent throug the latest sensing event.
The most notable data structures in this class are the following.

2.6.1 Parcel Reward Decay Frequency

In order to make precise and useful utlity value computations for both parcel pickups and deliveries, we make use of two things. The decay interval given by the server, stored in milliseconds, and the movement duration, also given by the server through the onConfig sensing event. With these, we compute a 'decay per move' value, which is a good estimate for the reward loss during one movement.
We use the following formula.

$$decayPerMove = \begin{cases} \frac{movementDuration}{decayInterval} & \text{if } decayInterval > 0 \\ 0 & \text{otherwise} \end{cases}$$

2.6.2 VisiblePositions

This set contains all visible positions through the latest sensing event.

2.6.3 Spawners

This map contains all known spawner tiles, a timestamp indicating when they've been last observed, and a boolean indicating if an operational delivery tile can be reached from each one of them. These values are indexed by a string indicating their coordinates.
An operational delivery tile is one that, when reached, doesn't lead to a deadlock. We added this boolean to deal with edge cases in some maps where some spawner or delivery tiles led to a deadlock once reached. This makes the agent avoid going to deliveries or spawners that would end its match.

2.6.4 Deliveries

This map is the equivalent of the 'Spawners' map, but it holds information about the known delivery tiles, paired with a boolean indicating if an operational spawner can be reached from there.

3 Desires and Intention Loop

As prescribed by the BDI architecture, the agent consults its beliefs and computes a set of desires, along with a utility value representing their potential reward if accomplished. The best option, that is the one with the highest utility value, is chosen and it becomes the current intention.
Periodically, the current intention gets revisioned with newer observations and game events, as we'll explain in another section.

3.1 Desires Generation

The generation of desires makes use of the agent's current and past observations, as described in the Belief section. The partner's shared information is also considered.
During generation, the desires are stored in an array containing objects representing the desires and the information needed to perform, evaluate and compare them.
These objects contain the following fields:

- type: it identifies the action to be performed by a given desire. It's a simple string of the form 'go_pick_up' or 'go_deliver'.
- target: the coordinates (x,y) of the desire's destination.
- utility: the utility value computed for the desire.
- parcel_id: for the desire of picking up percels, the parcels' id are also added to the object.
- delivery_distance: for the desire of delivering one or more parcels, the distance to the delivery coordinates, found in the target field, is added to the object.

We decided to develop a small, but efficient number of possible desires. We defined the following types of desires.

- go_pick_up: it defines the desire of picking up a parcel.
- go_deliver: it defines the desire of deliverying the currently carried parcels.
- go_to_spawner: this type of desire is used as an exploration desire. It is designed to be generated only when the agent currently doesn't have the option of either deliverying or picking up parcels.

3.1.1 Distances

To compute the utility values of our desires, we take into account the distances to reach the desires' target tiles. This is important as the game server can enable a parcel reward decay over time. This must be taken into account when evaluating which desire to perform, as further targets lead to likely smaller rewards as the parcels the agent is carrying and the parcels on the ground progressively lose more and more value.
To compute these distances, we've made use of the Breadth First Search (BFS). We opted for a BFS search instead of heuristical search algorithms such as A* due to the fact that movement costs are all equal. When costs are equal, a BFS search is optimal. This allows the agent to both find the shortest path to a target and to compute the utility values under optimal assumptions.

3.1.2 Pick-up Utility

This desire can be generated when the agent knows about at least one parcel that isn't currently being carried by anyone.
To generate this desire, we compute an utility score for each batch of parcels, a batch can be composed by only one parcel. We don't exclude this computation to a single parcel as multiple parcels can lay on a single tile, forming a batch whose utility score must be a sum of the utility of all its parcels. In addition, a pickup action picks up all the parcels in a given tile.
To compute the utility value of batches, we decided to also take into account the distance to deliver the picked up parcels from their location.
The distance used is the following.

$$totalDistance = distanceToBatch + distanceAfterPickup$$

Where 'distanceAfterPickup' is the distance to the closest delivery tile with respect to the batch's location.
This allows the agent to have an accurate and complete estimation of the whole process, it effectively gives it a prediction of the potential reward for picking up and then delivering one or more parcels. It also allows the agent to have a good measurement of comparison for utility of all the batches it can consider to pickup.
A batch's pickup utility score is given by the following formula.

$$batchPickupUtility = \sum_{p \in batch} \max(0, p.reward) - (decayPerMove * totalDistance)$$

It computes an estimation of the batch's reward once it has been picked up and the delivery tile is reached, taking decay into account.
To compute the pickup action's utlity, we also take into account the estimated reward of the parcels the agent is currently carrying, as their reward is gonna decay throughout the whole process.
The carried parcels' estimated reward is the following.

$$carriedReward = \sum_{p \in carried} \max(0, p.reward) - (moveDecay * distanceToDelivery)$$

For each pickup action option utility, we then use this formula.

$$pickupUtility = \frac{carriedReward + batchPickupUtility}{\max(1, totalDistance)}$$

The pickup option that gets inserted into the desires array is the one with the highest utility value.

3.1.3 Put-down Utility

This desire can be generated if the agent is currently carrying at least 1 parcel.
To generate this desire, we scan through all the delivery tiles in the game map. For each delivery tile, we compute the shortest path's distance to it, if it's reachable. If the given delivery tile is reachable, we compute its utility value.
We compute the delivery utility with the following formula:

$$deliveryUtility = \frac{expectedDeliveryReward}{\max(1, distanceToDelivery)}$$

Where 'expectedDeliveryReward' is the expected reward for delivering the parcels currently carried by the agent taking distance and reward decay into account. To compute this expected reward, we iterate through all carried parcels and compute an estimation of their decayed reward value once the agent reaches the delivery tile.
The formula is the following.

$$expectedDeliveryReward = \sum_{p \in carried} \max(0, p.reward) - (moveDecay * distanceToDelivery)$$

3.1.4 Exploration Utility

This desire is generated only when there is no other desirable action for the agent to perform. This happens when it has no parcels to deliver or to pickup, so the only option is to explore towards spawner tiles currenlty out of vision.
To compute the utility value of this desire, we iterate through the out of vision spawners, giving priority to operational ones. We then compute their exploration utility value.
This value is computed with the following formula.

$$spawnerExplorationUtility = movesSinceCheck / \max(1, Distance)$$

Where 'movesSinceCheck' is the amount of moves since the last time the agent saw the given spawner tile. This allows the agent to prioritise spawners it hasn't checked recently, as it's likely their state has changed.

3.1.5 Multipliers

The game server can assign missions to individual agents, teams or all players. These missions can assign reward multipliers for delivering stacks of parcels or for delivering in a certain delivery tile, among other cases. These multipliers are applied if needed after the utility values are computed and before they're inserted into the desires array.

3.2 Intention Loop and Intention Revision

[Insert text explaining intention revision.]

4 Planning

[Describe the pathfinding and action planning.]

4.1 PDDL

[Insert text.]

5 LLM Agent

[Describe the multi-agent communication protocol.]

6 Results and Conclusions

[Present the results of the project.]

7 Conclusions and future improvements

[Conclude the report and propose future developments.]